

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: Length Mock Exam Set-6(60 Questions)

Total Questions : 24

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: Full Length Mock Exam Set-6(60 Questions)

Questions Question - 1

What is output of the below code ?

```
public class A{
    private void test1(){

        System.out.println("test1 A");
    }
}

public class B extends A{
    public void test1(){

        System.out.println("test1 B");
    }
}

public class outputtest{
    public static void main(String[] args){
        A b = new B();
        b.test1();
    }
}
```

- 1.test1 A
- 2.test1 B
- 3.Not compile because test1() method in class A is not visible.
- 4.None of the above.

Explanation :

C is the correct answer.

Not compile because test1() method in class A is not private so it is not visible.

Question - 2

What is output of the below code ?

```
public class A{
    private void test1(){

        System.out.println("test1 A");
    }
}
```

```

public class B extends A{
    public void test1(){

        System.out.println("test1 B");
    }
}

public class Test{
    public static void main(String[] args){
        B b = new B();
        b.test1();

    }
}

```

- 1.test1 B
- 2.test1 A
- 3.Not compile because test1() method in class A is not visible
- 4.None of the above

Explanation :

A is the correct answer.

This is not related to superclass , B class object calls its own method so it compile and output normally.

Question - 3

Fill the gap

```

public class Test {

    public static void main(String... args) throws Exception {
        <String> lst = new LinkedList<String>();
        lst.add("one");
        lst.add("two");
        System.out.println(lst.poll());

    }
}

```

Use the following fragments zero or many times

Map

Sequence

Queue

Question - 4

Which of the following statements are true?

1. At the root of the collection hierarchy is a class called Collection
2. The collection interface contains a method called enumerator
3. The iterator method returns an instance of the Vector class
4. The Set interface is designed for unique elements

Explanation :

D is the correct answer.

Collection is an interface, not a class. The Collection interface includes a method called iterator. This returns an instance of the Iterator class which has some similarities with Enumerators. The name set should give away the purpose of the Set interface as it is analogous to the Set concept in relational databases which implies uniqueness.

Question - 5

How does the set collection deal with duplicate elements?

1. An exception is thrown if you attempt to add an element with a duplicate value
2. The add method returns false if you attempt to add an element with a duplicate value
3. A set may contain elements that return duplicate values from a call to the equals method
4. Duplicate values will cause an error at compile time

Explanation :

B is the correct answer.

The add method returns false if you attempt to add an element with a duplicate value

Question - 6

What will happen when you attempt to compile and run the following code?

```
import java.io.*;
class ExBase{
    abstract public void test(){
    }
}

public class MyEx extends ExBase{
    public static void main(String argv[]){
        DataInputStream fi = new DataInputStream(System.in);
        try{
            fi.readChar();
        }catch(IOException e){
```

```

        System.exit(0);
    }
    finally {System.out.println("Doing finally");}
}

```

- 1.Compile with error
- 2.It will run, wait for a key press and then exit
- 3.It will run, wait for a keypress, print "Doing finally" then exit
- 4.At run and immediately exit

Explanation :

A is the correct answer.

It will produce an error like "Abstract and native method can't have a body". This is typical of the more misleading question where you might think it is asking you about the circumstances under which the finally clause runs, but actually it is about something else.

Question - 7

What is the output ?

```

public class Test {
    public static void main(String... args) {
        final int i;
        System.out.println(i);
    }
}

```

- 1.Compile with error
- 2.0
- 3.Runtime Exception
- 4.None of the above

Explanation :

A is the correct answer.

final variables should be initialized.

Question - 8

```

interface I{
    void f1();           // 1
    public void f2();    // 2
    protected void f3(); // 3
    private void f4();   // 4
}

```

- 1.compiletime error at lines 1,2,3,4
- 2.compiletime error at line 3
- 3.compiletime error at line 1
- 4.compiletime error at lines 3,4

Explanation :

D is the correct answer.

all methods declared within an interface are implicitly public, a weaker access level can not be declared.

Question - 9

What is the result when you compile and run the following code?

```
public class Test
{
    static void throwMethod()
    {
        System.out.println("Inside throwMethod.");
        throw new IllegalAccessException("demo");
    }

    public static void main(String args[])
    {
        try
        {
            throwMethod();
        }
        catch (IllegalAccessException e)
        {
            System.out.println("Caught " + e);
        }
    }
}
```

- 1.Compile with error
- 2.Runtime error
- 3.Compile successfully, nothing is printed.
- 4.Inside throwMethod. followed by caught: java.lang.IllegalAccessException: demo

Explanation :

A is the correct answer.

static void throwMethod() should throw IllegalAccessException.

Question - 10

The relationship between a class and its superclass is called _____

- 1.has-a
- 2.is -a
- 3.None of the above
- 4.none of the above

Explanation :

B is the correct answer.

The relationship between a class and its superclass is called is -a relationship.

Question - 11

```
class Test {
    public static void main(String args[]) throws
InterruptedException {
        Test t1 = new Test();
        Test t2 = m1(t1);
        Test t3 = new Test();
        Test t4 = new Test();
        t4 = null;
        t2 = t3; // 6
        anotherMethod();
    }

    static Test m1(Test ob1) {
        ob1 = new Test();
        return ob1;
    }
}
```

After line 6, how many objects are eligible for garbage collection?

- 1.1
- 2.2
- 3.3
- 4.4

Explanation :

D is the correct answer.

Total 4 objects are created . So 4 objects will be garbage collected. You can do it by yourself class Test { public static void main(String args[]) throws InterruptedException { Test t1 = new Test(); System.out.println("t1 : "+t1.hashCode()); Test t2 = m1(t1); System.out.println("t2 : "+t2.hashCode()); Test t3 = new Test(); System.out.println("t3 : "+t3.hashCode()); Test t4 = new Test(); System.out.println("t4 : "+t4.hashCode()); t4 = null; t2 = t3; // 6 Thread.sleep(2000); System.gc(); Thread.sleep(5000); // anotherMethod(); } public void finalize() { System.out.println("Garbage Collected : "+this.hashCode()); } static Test m1(Test ob1) { ob1 = new Test(); System.out.println("ob1 : "+ob1.hashCode()); return ob1; } }

Question - 12

What is the output?

```
class Bird {  
    boolean b;  
  
    public Bird() {  
        System.out.println(b);  
    }  
public static void main(String[] args){  
    Bird b = new Bird();  
}  
}
```

- 1.true
- 2.false
- 3.garbage
- 4.none of the above

Explanation :

B is the correct answer.

default value of boolean is false.

Question - 13

Which most closely matches a description of a Java Map?

- 1.A vector of arrays for a 2D geographic representation
- 2.A class for containing unique array elements
- 3.A class for containing unique vector elements
- 4.An interface that ensures that implementing classes cannot contain duplicate keys

Explanation :

D is the correct answer.

Map does not have any duplicate keys.

Question - 14

What will be output by the following line?

```
System.out.println(Math.floor(-2.1));
```

- 1.-2.0
- 2.-3.0
- 3.2.0

4.3.0

Explanation :

B is the correct answer.

floor method return the largest (closest to positive infinity) floating-point value that less than or equal to the argument and is equal to a mathematical integer.

Question - 15

What will be the result when you try to compile and run the following code?

```
public class Base {
    Base() {
        int i = 100;
        System.out.println(i);
    }
}

public class Sub extends Base{

    static int i = 200;
    public static void main(String argv[]){
        Sub s = new Sub();
        System.out.println(i);
    }
}
```

- 1.Error at compile time
- 2.200
- 3.100 followed by 200
- 4.100

Explanation :

C is the correct answer.

On execution Sub s = new Sub(); super class constructor will be called because Sub extends Base.

Question - 16

```
int count=0, i=0;
do {
    count *= i;
    i++;
    if(count == 1) continue;
    count++;
} while(i < 3);
System.out.println(count);
```

What is the output?

- 1.4
- 2.2
- 3.3
- 4.5

Explanation :

C is the correct answer.

trace it by yourself.

Question - 17

```
public class Sub{  
  
    public static void main(String argv[]){  
  
        byte b1=33;           //1  
        b1++;                  //2  
        byte b2=55;           //3  
        b2=b1+1;               //4  
        System.out.println(b1+" "+b2) ;  
  
    }  
}
```

- 1.compile time error at line 2
- 2.compile time error at line 4
- 3.prints 34,56
- 4.runtime exception

Explanation :

B is the correct answer.

b1+1 returns an integer value which can not be assigned to a byte variable. Type mismatch: cannot convert from int to byte

Question - 18

The Throwable class is the superclass of all exceptions in the Java language.

- 1.true
- 2.false
- 3.none of the above
- 4.none of the above

Explanation :

A is the correct answer.

Throwable class is the superclass of all exceptions

Question - 19

Which of the following most closely describes the process of overriding?

1. A class with the same name replaces the functionality of a class defined earlier in the hierarchy
2. A method with the same name completely replaces the functionality of a method earlier in the hierarchy
3. A method with the same name but different parameters gives multiple uses for the same method name
4. A class is prevented from accessing methods in its immediate ancestor

Explanation :

B is the correct answer.

Option 3 is more like a description of overloading. I like to remind myself of the difference between overloading and overriding in that an overridden method is like something overridden in the road, it is squashed, flat no longer used and replaced by something else. An overloaded method has been given extra work to do (it is loaded up with work), but it is still being used in its original format. This is just my little mind trick and doesn't match to anything that Java is doing

Question - 20

What results from the following code?

```
1.  class MyClass
2.  {
3.      void myMethod(int i) {System.out.println("int version");}
4.      void myMethod(String s) {System.out.println("String
version");}
5.      public static void main(String args[])
6.      {
7.          MyClass obj = new MyClass();
8.          char ch = 'c';
9.          obj.myMethod(ch);
10.     }
11. }
```

1. Line 4 will not compile as void methods can't be overridden.
2. An exception at line 9.
3. Line 9 will not compile as there is no version of myMethod which takes a char as argument.

4.The code compiles and produces output: int version.

Explanation :

D is the correct answer.

if same method parameter type id not found then try to findout nearest type. The method resolution includes three passes: 1. Attempt to locate the correct method without any boxing, unboxing, or vararg invocations. 2. Attempt to locate the correct method with boxing, unboxing, and without any vararg invocations. 3. Attempt to locate the correct method with boxing, unboxing, or vararg invocations.

Question - 21

```
public class Sub{  
  
    public static void main(String argv[]){  
  
        String s1 = "abc";  
        StringBuffer sb2 = new StringBuffer(s1);  
        System.out.print(sb2.equals(s1) + "," + s1.equals(sb2));  
  
    }  
}
```

- 1.Prints: false,false
- 2.Prints: true,false
- 3.Prints: false,true
- 4.Prints: true,true

Explanation :

A is the correct answer.

s1 and sb2 are pointing to different object references

Question - 22

What is the value of p?

```
public class Sub{  
  
    public static void main(String argv[]){  
  
        int p = 10 / 3;  
        System.out.println(p);  
  
    }  
}
```

- 1.3
- 2.3.33333

3.0
4.1

Explanation :

A is the correct answer.

p is int so output is 3.

Question - 23

What is the output for the below code ?

```
public class Sub{  
  
    public static void main(String argv[]){  
  
        Math m = new Math();  
        System.out.println(m.max(10,15));  
  
    }  
}
```

1.Compile with error
2.10
3.15
4.0

Explanation :

A is the correct answer.

constructor of Math class is private So compiler error.

Question - 24

You want to find out the value of the last element of an array. You write the following code. What will happen when you compile and run it.?

```
public class MyAr{  
    public static void main(String argv[]){  
        int[] i = new int[5];  
        System.out.println(i[5]);  
    }  
}
```

1.An error at compile time
2.An error at run time
3.The value 0 will be output
4.The string "null" will be output

Explanation :

B is the correct answer.

java.lang.ArrayIndexOutOfBoundsException because array start from index 0 , i[4] will return the fifth element.

Question - 25

Which of the following best describes the use of the synchronized keyword?

- 1.Allows two process to run in paralell but to communicate with each other
- 2.Ensures only one thread at a time may access a method or object
- 3.Ensures that two or more processes will start and end at the same time
- 4.Ensures that two or more Threads will start and end at the same time

Explanation :

B is the correct answer.

synchronized : Ensures only one thread at a time may access a method or object.

Question - 26

Which of the following statements are true?

- 1.Static methods cannot be overridden by non static method
- 2.Static methods cannot be declared as private
- 3.Private methods cannot be overloaded
- 4.An overloaded method cannot throw exceptions not checked in the base class

Explanation :

A is the correct answer.

Static methods cannot be overridden by non static method.

Question - 27

What is the result when you compile and run the following code?

```
public class Test
{
    public void method()
    {
        for(int i = 0; i < 3; i++)
        {
            System.out.print(i);
        }
    }
}
```

```

        }
        System.out.print(i) ;
    }
}

```

- 1.0122
- 2.0123
- 3.Compilation error
- 4.None of these

Explanation :
C is the correct answer.

variable i is not visible outside for loop.

Question - 28

```

class C {
public static void main(String[] args) {
    int i1=1;
    switch(i1){
        case 1:
            System.out.println("one") ;

        case 2:
            System.out.println("two") ;

        case 3:
            System.out.println("three") ;
    }}
}

```

- 1.prints one two three
- 2.prints one
- 3.compile time error
- 4.Runtime exceptionf

Explanation :
A is the correct answer.

There is no break statement in case 1 so it causes the below case statements to execute regardless of their values

Question - 29

```

abstract class vehicle{
abstract public void speed() ;
}
class car extends vehicle{
public static void main (String args[]) {
vehicle ob1;
ob1=new car() ;    //1
}
}

```

```
}}
```

- 1.compiletime error at line 1
- 2.forces the class car to be declared as abstract
- 3.Runtime Exception
- 4.None of the above

Explanation :

B is the correct answer.

if you extend a abstract class , you have to implement the abstract method or make sub class also abstract.

Question - 30

Given the following class

```
public class Sub{  
  
    public static void main(String argv[]){  
        boolean b1 = true;  
        if((b1 ==true) || istrue(true)){  
            System.out.println("abc");  
        }  
    }  
  
    public static boolean istrue(boolean location){  
        if(location==true){  
            System.out.println("pqr");  
        }  
        System.out.println("xyz");  
        return true;  
    }  
}
```

What will happen when you attempt to compile and run it?

- 1.Compile with error
- 2.prints abc
- 3.prints pqr xyz abc
- 4.prints xyz abc

Explanation :

B is the correct answer.

This code is an example of a short circuited operator. Because the first operand of the || (or) operator returns true Java sees no reason to evaluate the second. Whatever the value of the second the overall result will always be true. Thus the method called istrue is never called.

Question - 31

Which of the following statements are true?

- 1) All of the variables in an interface are implicitly static
- 2) All of the variables in an interface are implicitly final
- 3) All of the methods in an interface are implicitly abstract
- 4) A method in an interface can access class level variables

1.1 ,2 and 3

2.1 ,2 ,3 and 4

3.1 and 4

4.2 and 4

Explanation :

A is the correct answer.

All the variables in an interface are implicitly static and final. Any methods in an interface have no body, so may not access any type of variable

Question - 32

```
public interface Int { //1
    public class Inner { //2
        Inner ( ) {
            System.out.println ( "Inner Created" ) ;
        }
    };
}
```

Which of the below statement is true about above code ?

- 1.compile time error at line 1
- 2.compile time error at line 2
- 3.the code compiles fine
- 4.none of the above

Explanation :

C is the correct answer.

No Explanation Available

Question - 33

```
class C {
```

```
public static void main(String[] args) {  
    double d1 = Math.floor(0.5);  
    double d2 = Math.floor(1.5);  
    System.out.print(d1 + "," + d2);  
}}
```

1. Prints: 0.0,1.0
2. Prints: 0.0,2.0
3. Prints: 1.0,1.0
4. Prints: 1.0,2.0

Explanation :

A is the correct answer.

Returns the largest (closest to positive infinity) double value that is not greater than the argument and is equal to a mathematical integer.

Question - 34

Which of the following is not Java modifier?

- 1) public
- 2) private
- 3) friendly
- 4) transient

- 1.1
- 2.2
- 3.3
- 4.4

Explanation :

C is the correct answer.

friendly is not Java modifier.

Question - 35

A signed data type has an equal number of non-zero positive and negative values available

1. true
2. false
3. none of the above
4. none of the above

Explanation :

B is the correct answer.

The range of negative numbers is greater by 1 than the range of positive numbers

Question - 36

```
class bike
{
}
class arr extends bike{
public static void main(String[] args) {
arr[] a1=new arr[2];
bike[] a2;
a2=a1;                //3
arr[] a3;
a3=a1;                //5
}}
```

- 1.compile time error at line 3
- 2.compile time error at line 5
- 3.Runtime exception
- 4.The code runs fine

Explanation :

D is the correct answer.

bike is the superclass of arr.so they are compatible(superobject=subobject) but subobject=superobject not allowed

Question - 37

What will be the result of compiling the following code:

```
public class SuperClass {
    public int doIt(String str, Integer... data){
        String signature = "(String, Integer[])";
        System.out.println(str + " " + signature);
        return 1;
    }
}

public class SubClass extends SuperClass{

    public int doIt(String str, Integer... data)
    {
        String signature = "(String, Integer[])";
        System.out.println("Overridden: " + str + " " + signature);
        return 0;
    }

    public static void main(String[] args) throws Exception {
        SuperClass ref = new SubClass();
        ref.doIt("1. (String)",30);
    }
}
```

```
    }  
}
```

- 1.The code does not compile.
- 2.The code compiles cleanly and print Overridden: 1. (String) (String, Integer[])
- 3.The code throws an Exception at Runtime.
- 4.None of the above

Explanation :

B is the correct answer.

Overriding of varargs methods does not present any surprises as long as criteria for overriding is satisfied

Question - 38

What will be the result of compiling the following code:

```
public class SuperClass {  
    public int doIt(String str, Integer... data){  
        String signature = "(String, Integer[])";  
        System.out.println(str + " " + signature);  
        return 1;  
    }  
}  
  
public class SubClass extends SuperClass{  
  
    public int doIt(String str, Integer... data)  
    {  
        String signature = "(String, Integer[])";  
        System.out.println("Overridden: " + str + " " + signature);  
        return 0;  
    }  
  
    public static void main(String[] args) throws Exception {  
        SuperClass ref = new SubClass();  
        ref.doIt("1. (String)",30,40,60);  
    }  
}
```

- 1.The code does not compile.
- 2.The code compiles cleanly and print Overridden: 1. (String) (String, Integer[])
- 3.The code throws an Exception at Runtime.
- 4.None of the above

Explanation :

B is the correct answer.

Overriding of varargs methods does not present any surprises as long as criteria for overriding is satisfied

Question - 39

Which bellow statement is true about Varargs ?

- 1.Varargs can be used only in the final argument position
- 2.Varargs can be used any argument position
- 3.Varargs can be only used as String arguments
- 4.None of the above

Explanation :

A is the correct answer.

Varargs can be used only in the final argument position. like public static String format(String pattern,Object... arguments);

Question - 40

What is the output ?

```
public class Test {  
  
    public void add(int... a){  
        System.out.println("add int");  
    }  
    public void add(int... a,double b){  
        System.out.println("add int and double");  
    }  
    public static void main(String... args) {  
        Test t = new Test();  
        t.add(2);  
    }  
}
```

- 1.add int
- 2.add int and double
- 3.Compile error
- 4.None of the above

Explanation :

C is the correct answer.

Compile error : Varargs can be used only in the final argument position

Question - 41

What is the output for the bellow code ?

```
public class Test {
    enum Day {
        MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY,
        SUNDAY
    }

    public static void main(String[] args) {

        int[] freqArray = { 12, 34, 56, 23, 5, 13, 78 };

        // Create a Map of frequencies
        Map<Day, Integer> ordinaryMap = new HashMap<Day, Integer>();
        for (Day day : Day.values()) {
            ordinaryMap.put(day, freqArray[day.ordinal()]);
        }

        // Create an EnumMap of frequencies
        EnumMap<Day, Integer> frequencyEnumMap = new EnumMap<Day,
Integer>(ordinaryMap);

        // Change some frequencies
        frequencyEnumMap.put(Day.MONDAY, 100);
        frequencyEnumMap.put(Day.FRIDAY, 123);
        System.out.println("Frequency EnumMap: " + frequencyEnumMap);

    }
}
```

1. Frequency EnumMap: {MONDAY=100, TUESDAY=34, WEDNESDAY=56, THURSDAY=23, FRIDAY=123, SATURDAY=13, SUNDAY=78}
2. Frequency EnumMap: {MONDAY=12, TUESDAY=34, WEDNESDAY=56, THURSDAY=23, FRIDAY=5, SATURDAY=13, SUNDAY=78}
3. Compile with error
4. None of the above

Explanation :

A is the correct answer.

// Creates an empty enum map with the specified key type. EnumMap(Class keyType)
// Creates an enum map with the same key type as the specified enum map, initially //
containing the same mappings (if any). EnumMap(EnumMap m) // Creates an enum
map initialized from the specified map. EnumMap(Map m)

Question - 42

What is the output for bellow code ?

```
public class Test {
    enum Day {
```

```

        MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY,
SUNDAY
    }

    public static void main(String[] args) {

        int[] freqArray = { 12, 34, 56, 23, 5, 13, 78 };

        // Create a Map of frequencies
        Map<Day, Integer> ordinaryMap = new HashMap<Day, Integer>();
        for (Day day : Day.values()) {
            ordinaryMap.put(day, freqArray[day.ordinal()]);
        }

        // Create an EnumMap of frequencies
        EnumMap<Day, Integer> frequencyEnumMap = new EnumMap<Day,
Integer>(ordinaryMap);

        // Change some frequencies
        frequencyEnumMap.put("abc", 100);
        frequencyEnumMap.put("bcd", 123);
        System.out.println("Frequency EnumMap: " + frequencyEnumMap);

    }

}

```

1. Frequency EnumMap: {MONDAY=100, TUESDAY=34, WEDNESDAY=56, THURSDAY=23, FRIDAY=123, SATURDAY=13, SUNDAY=78}
2. Frequency EnumMap: {abc=100,bcd=123,MONDAY=100, TUESDAY=34, WEDNESDAY=56, THURSDAY=23, FRIDAY=123, SATURDAY=13, SUNDAY=78}
3. Compile with error
4. None of the above

Explanation :

C is the correct answer.

All of the keys in an enum map MUST come from a single enum type.

Question - 43

What is the output for the bellow code ?

```

public class Test {
    enum Day {
        MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY,
SUNDAY
    }
    enum Month {
        JAN, FEB
    }
}

```

```

public static void main(String[] args) {

    int[] freqArray = { 12, 34, 56, 23, 5, 13, 78 };

    // Create a Map of frequencies
    Map<Day, Integer> ordinaryMap = new HashMap<Day, Integer>();
    for (Day day : Day.values()) {
        ordinaryMap.put(day, freqArray[day.ordinal()]);
    }

    // Create an EnumMap of frequencies
    EnumMap<Day, Integer> frequencyEnumMap = new EnumMap<Day,
Integer>(ordinaryMap);

    // Change some frequencies
    frequencyEnumMap.put(Month.JAN, 100);

    System.out.println("Frequency EnumMap: " + frequencyEnumMap);

}
}

```

1. Frequency EnumMap: {JAN=100, MONDAY=100, TUESDAY=34, WEDNESDAY=56, THURSDAY=23, FRIDAY=123, SATURDAY=13, SUNDAY=78}
2. Compile with error
3. Runtime Exception
4. None of the above

Explanation :

B is the correct answer.

All of the keys in an enum map MUST come from a single enum type. EnumMap is trying to add key from both the enums (Day and Month).

Question - 44

Under what circumstances might you use the yield method of the Thread class

1. To call from the currently running thread to allow another thread of the same or higher priority to run
2. To call on a waiting thread to allow it to run
3. To allow a thread of higher priority to run
4. To call from the currently running thread with a parameter designating which thread should be allowed to run

Explanation :

A is the correct answer.

Option 3 looks plausible but there is no guarantee that the thread that grabs the cpu time will be of a higher priority. It will depend on the threading algorithm of the Java Virtual Machine and the underlying operating system

Question - 45

Which of the following statements about threading are true

- 1) You can only obtain a mutually exclusive lock on methods in a class that extends Thread or implements runnable
- 2) You can obtain a mutually exclusive lock on any object
- 3) A thread can obtain a mutually exclusive lock on an object by calling a synchronized method on that object.
- 4) Thread scheduling algorithms are platform dependent

- 1.2 , 3 and 4
- 2.1 and 2
- 3.1 , 3 and 4
- 4.None of the above

Explanation :

A is the correct answer.

Yes that says dependent and not independent.

Question - 46

Which of the following best describes the use of the synchronized keyword?

- 1.Allows two process to run in paralell but to communicate with each other
- 2.Ensures only one thread at a time may access a method or object
- 3.Ensures that two or more processes will start and end at the same time
- 4.Ensures that two or more Threads will start and end at the same time

Explanation :

B is the correct answer.

Ensures only one thread at a time may access a method or object

Question - 47

What is the output ?

```

public class Test extends Thread{

    static String sName = "good";
    public static void main(String argv[]){
        Test t = new Test();
        t.piggy(sName);
        System.out.println(sName);

    }
    public void piggy(String sName){
        sName = sName + " idea ";
        start();
    }
    public void run(){

        for(int i=0;i < 4; i++){
            sName = sName + " " + i;

        }
    }
}

```

- 1.good
- 2.good idea
- 3.good idea good idea
- 4.None of the above

Explanation :

A is the correct answer.

You cannot be certain of this though.

Question - 48

Is the bellow statement is true about static import ?

"static import declaration imports static members from classes, allowing them to be used without class qualification"

- 1.true
- 2.false
- 3.None of the above
- 4.None of the above

Explanation :

A is the correct answer.

"static import declaration imports static members from classes, allowing them to be used without class qualification" Example package bean; public class Abc { public static int index_val = 10; } package com; import static bean.Abc.index_val; public class Test1 { public static void main(String... args)

{ System.out.println(index_val); } } Do import static bean.Abc.index_val; and You can use index_val directly ..

Question - 49

What is the output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        Pattern p = Pattern.compile("a{1,3}b?c*");  
        Matcher m = p.matcher("aaaab");  
        boolean b = m.matches();  
        System.out.println(b);  
  
    }  
}
```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

B is the correct answer.

X? X, once or not at all X* X, zero or more times X+ X, one or more times X{n} X, exactly n times X{n,} X, at least n times X{n,m} X, at least n but not more than m times

Question - 50

What is the output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        Pattern p = Pattern.compile("[,\\s]+");  
  
        String[] result = p.split("one,two, three four , five");  
        for (int i=0; i<result.length; i++) {  
            System.out.println( result[i] );  
        }  
  
    }  
}
```

- 1.one two three four five
- 2.one two three five
- 3.Compile error
- 4.None of the above

Explanation :

A is the correct answer.

[,\s]+ means comma or white space one or more time so split based on the criteria.

Question - 51

Which statement is true?

- 1.an inner class instance can never stand alone without a direct relationship to an instance of the outer class
- 2.you can access the inner class is through a live instance of the outer class
- 3.Inner class can access private member of the outer class.
- 4.None of the above

Explanation :

A, B and C is the correct answer.

an inner class instance can never stand alone without a direct relationship to an instance of the outer class. you can access the inner class is through a live instance of the outer class. Inner class can access private member of the outer class.

Question - 52

```
public class Outer {
    private int a = 7;

    class Inner {
        public void displayValue() {
            System.out.println("Value of a is " + a);
        }
    }
}

public class Test {

    public static void main(String... args) throws Exception {
        Outer mo = new Outer();
        Outer.Inner inner = mo.new Inner();
        inner.displayValue();
    }
}
```

```
}
```

What is the output?

- 1.Value of a is 7
- 2.Compile Fail
- 3.Runtime Exception
- 4.None of the above

Explanation :

A is the correct answer.

an inner class instance can never stand alone without a direct relationship to an instance of the outer class. you can access the inner class is through a live instance of the outer class. Inner class can access private member of the outer class.

Question - 53

```
public class A {
    public A() {
        System.out.println("A");
    }
}

public class B extends A implements Serializable {
    public B() {
        System.out.println("B");
    }
}

public class Test {

    public static void main(String... args) throws Exception {
        B b = new B();

        ObjectOutputStream save = new ObjectOutputStream(new
FileOutputStream("datafile"));
        save.writeObject(b);
        save.flush();

        ObjectInputStream restore = new ObjectInputStream(new
FileInputStream("datafile"));
        B z = (B) restore.readObject();

    }
}
```

What is the output?

- 1.A B A
- 2.A B A B

3.B B

4.B

Explanation :

A is the correct answer.

On the time of deserialization , the Serializable object not create new object. So constructor of class B does not called. A is not Serializable object so constructor is called.

Question - 54

```
public class A {
    public A() {
        System.out.println("A");
    }
}

public class Test {

    public static void main(String... args) throws Exception {
        A a = new A();

        ObjectOutputStream save = new ObjectOutputStream(new
        FileOutputStream("datafile"));
        save.writeObject(a);
        save.flush();

        ObjectInputStream restore = new ObjectInputStream(new
        FileInputStream("datafile"));
        A z = (A) restore.readObject();

    }
}
```

What is the output?

1.A A

2.A

3.java.io.NotSerializableException

4.None of the above

Explanation :

C is the correct answer.

Class A does not implements Serializable interface. So throws NotSerializableException on trying to Serialize a non Serializable object.

Question - 55

```
public class A implements Serializable{
    public A() {
        System.out.println("A");
    }
}

public class Test {

    public static void main(String... args) throws Exception {
        A a = new A();

        ObjectOutputStream save = new ObjectOutputStream(new
        FileOutputStream("datafile"));
        save.writeObject(a);
        save.flush();

        ObjectInputStream restore = new ObjectInputStream(new
        FileInputStream("datafile"));
        A z = (A) restore.readObject();

    }
}
```

What is the output?

- 1.A A
- 2.A
- 3.Runtime Exception
- 4.Compile with error

Explanation :

B is the correct answer.

On the time of deserialization , the Serializable object not create new object. So constructor of class A does not called.

Question - 56

Is the bellow statement is true?

"ArrayIndexOutOfBoundsException is a type of RuntimeException"

- 1.true
- 2.false
- 3.None of the above
- 4.None of the above

Explanation :

A is the correct answer.

java.lang.Object java.lang.Throwable java.lang.Exception
java.lang.RuntimeException java.lang.IndexOutOfBoundsException
java.lang.ArrayIndexOutOfBoundsException

Question - 57

What is the output ?

```
public class Test {  
    public static void main(String... args) {  
  
        String strarr[] = new String[2] ;  
        System.out.println(strarr[2]);  
  
    }  
  
}
```

- 1.Compile with error
- 2.java.lang.ArrayIndexOutOfBoundsException
- 3.None of the above
- 4.None of the above

Explanation :

B is the correct answer.

String strarr[] = new String[2] has size 2 , and strarr[2] is trying to access third element. array starts with strarr[0]. So ArrayIndexOutOfBoundsException.

Question - 58

What is the output ?

```
public class Test {  
    public static void main(String... args) {  
  
        Object x = new Integer(0) ;  
        System.out.println((String)x) ;  
  
    }  
  
}
```

- 1.Compile with error
- 2.java.lang.ClassCastException

- 3. None of the above
- 4. None of the above

Explanation :

B is the correct answer.

trying to cast integer to String.

Question - 59

What will be the result of compiling the following code:

```
public class Test {  
    public static void main(String... args) {  
        Test t = new Test();  
        System.out.println(t.getName());  
    }  
  
    public String getName() throws Exception  
    {  
        return "das";  
    }  
}
```

- 1. the code will compile and print das
- 2. the code will compile and print null
- 3. Compile time error.
- 4. None of the above

Explanation :

C is the correct answer.

Unhandled exception type Exception. Throws exception should catch by caller.

Question - 60

What will be the result of compiling the following code:

```
public class SuperClass {  
    public int doIt(String str, Integer... data) throws Exception {  
        String signature = "(String, Integer[])";  
        System.out.println(str + " " + signature);  
        return 1;  
    }  
}  
  
public class SubClass extends SuperClass {
```

```

    public int doIt(String str, Integer... data)
    {
        String signature = "(String, Integer[])";
        System.out.println("Overridden: " + str + " " + signature);
        return 0;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();
        sb.doIt("hello", 3);
    }
}

```

- 1.Overridden: hello (String, Integer[])
- 2.hello (String, Integer[])
- 3.Compile time error.
- 4.None of the above

Explanation :

C is the correct answer.

Unhandled exception type Exception.